

École Nationale d'Intelligence Artificielle et du Digital Berkane
Université Mohammed Premier

Série de TP N° 1 - Conception et développement d'IHM

1ère partie : ES6

Partie abordées dans cette partie :

- Les fonctions de tableaux : Array.map(), Array.filter(), et Array.reduce()
- Les promesses
- Async/Await
- ES modules

Travail à faire :

1. Écrivez une fonction en utilisant Array.map() qui prend le tableau [1, 2, 3, 4, 5] de nombres en entrée et renvoie un nouveau tableau où chaque élément est le double de l'élément correspondant dans le tableau d'origine. (utiliser la syntaxe des fonctions fléchées)
2. Écrivez une fonction en utilisant Array.map() qui prend un tableau d'objets représentant des personnes avec des propriétés "prénom" et "nom" et renvoie un nouveau tableau de chaînes de caractères représentant le nom complet formaté.

Exemple d'entrée :

```
const personnes = [  
  { prenom: 'Ahmed', nom: 'ALAMI' },  
  { prenom: 'Fatima', nom: 'ALAOUI' },  
  { prenom: 'Karim', nom: 'BAGHDADI' }  
];
```

Résultat attendu : ['Ahmed ALAMI', 'Fatima ALAOUI', 'Karim BAGHDADI']

3. Écrivez une fonction en utilisant Array.filter() qui prend le tableau [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] de nombres en entrée et renvoie un nouveau tableau contenant uniquement les nombres pairs.
4. Écrivez une fonction en utilisant Array.reduce() qui prend un tableau de valeurs en entrée et renvoie un objet contenant la fréquence de chaque élément du tableau.
Exemple d'entrée : ['a', 'b', 'c', 'a', 'b', 'a', 'c', 'd']
Résultat attendu : { 'a': 3, 'b': 2, 'c': 2, 'd': 1 }
5. Soit la fonction chargerDonnees ci-dessous :

```
// Déclaration de la fonction pour charger les données à partir d'une API  
const chargerDonnees = url => {  
  return new Promise((resolve, reject) => {
```

```
    fetch(url)
      .then(response => {
        if (!response.ok) {
          throw new Error('Erreur lors du chargement des données');
        }
        return response.json();
      })
      .then(data => {
        resolve(data);
      })
      .catch(error => {
        reject(error.message);
      });
  });
};
```

Cette fonction charge des données à partir d'une API distante en utilisant **fetch** et retourne une promesse.

Consommer cette promesse en chargeant des données à partir de l'API (<https://jsonplaceholder.typicode.com/posts/1>) et affichez-les une fois qu'elles sont chargées, ou gérer les erreurs si la requête échoue.

6. Modifier le corps de la fonction `chargerDonnees()` pour profiter de la syntaxe `Async/await`.

7. a. Créez un fichier **operations.js** avec les opérations suivantes :

- Une fonction nommée **addition** qui prend deux nombres en entrée et retourne leur somme.
- Une fonction nommée **multiplication** qui prend deux nombres en entrée et retourne leur produit.

b. Créez un fichier **index.js** pour importer et utiliser les fonctions du fichier `operations.js`

- Importez les fonctions `addition` et `multiplication` du fichier `operations.js`
- Utilisez ces fonctions pour effectuer quelques opérations mathématiques dans le fichier `index.js`.